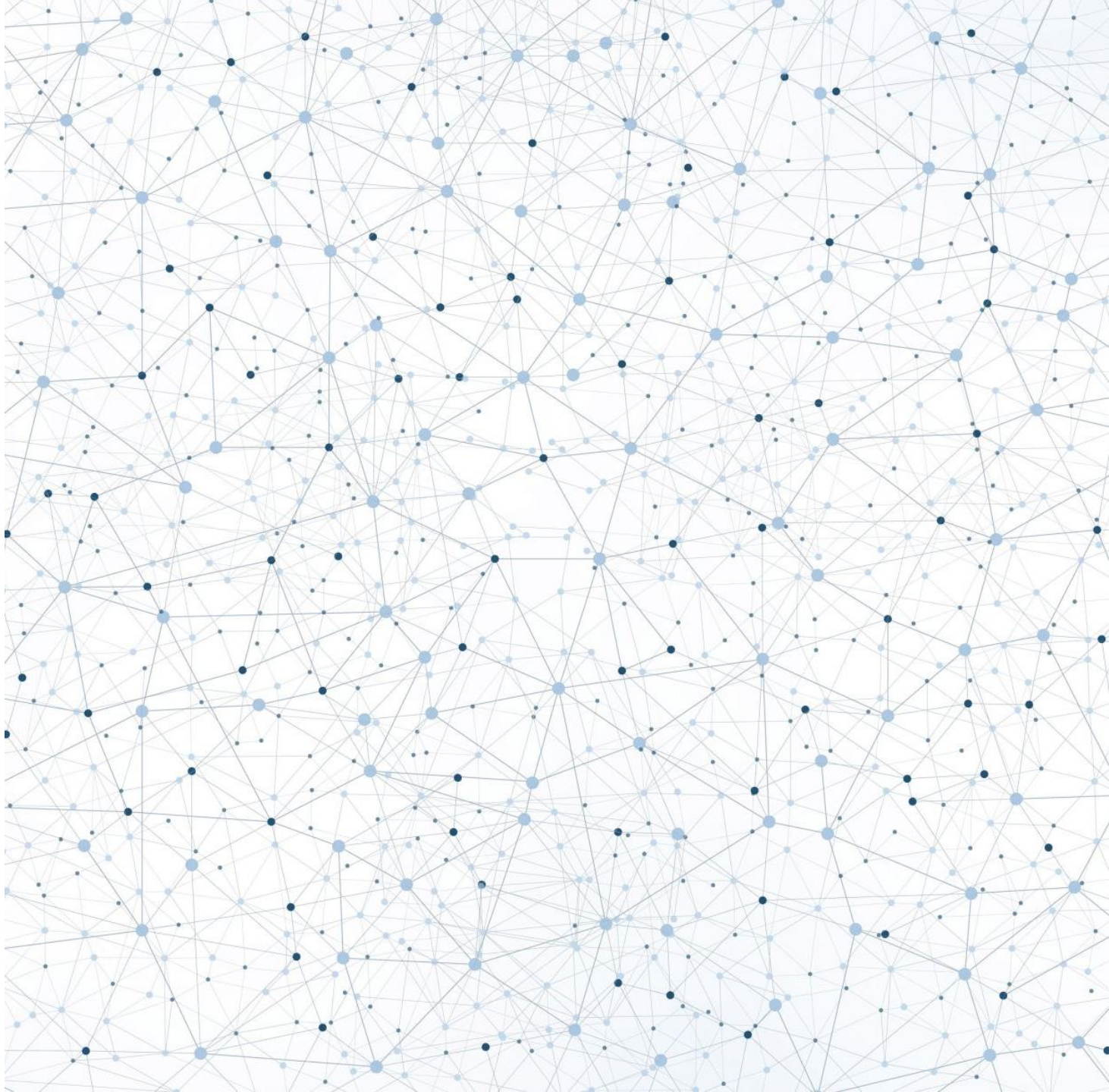
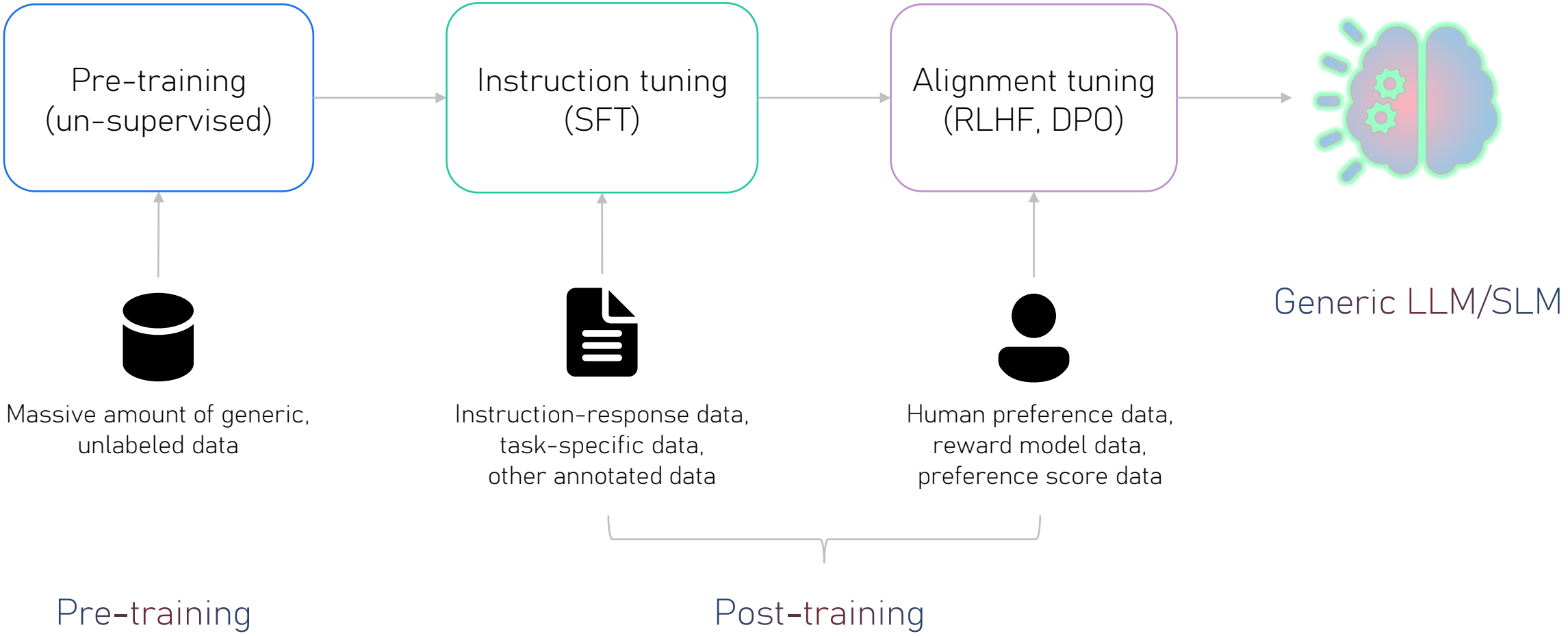
# Fine-Tuning Pre-Trained Models

(NER & CLASSIFICATION FOCUS)



# The "Why" and "What" of Fine-Tuning

# Building a foundation model




Watch BRK 150: Microsoft Build: https://www.youtube.com/watch?v=hw6-rrYtUy8

# Why fine-tune?

## Enhance performance and accuracy

- Domain-specific customization
- Task-specific optimization

## Cost reduction and efficiency

- Reduced token consumption
- Efficient resource utilization

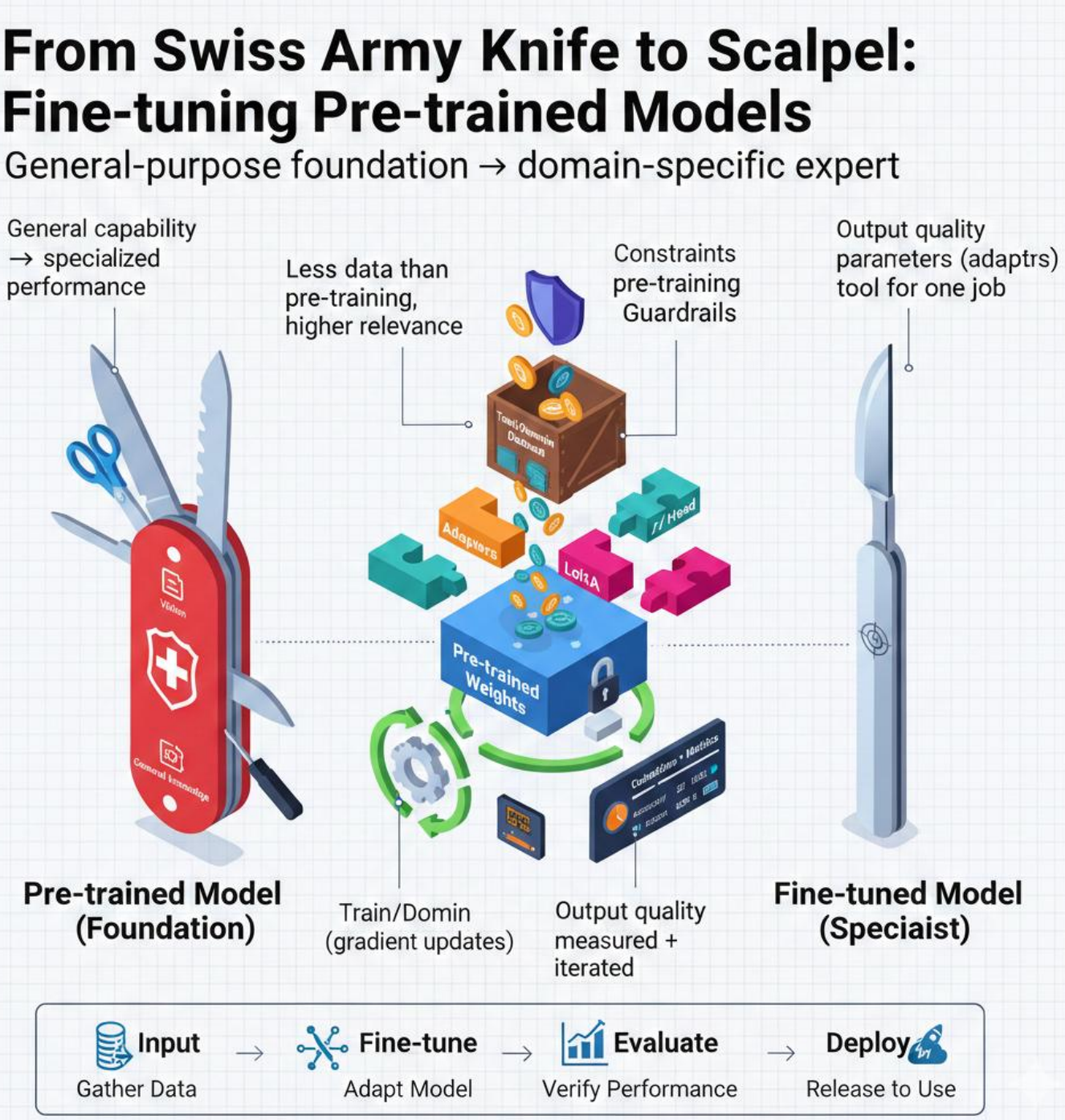
## Lower latency

- Smaller, faster models
- Fewer tokens – faster responses

## Risk mitigation

- Reduce bias and improve fairness
- Avoid hallucinations

BRK 150 deck: https://lnkd.in/ga7c2mj3 & Code: https://lnkd.in/g2-c-g_G




# Fine-Tuning Pre-Trained Models Turning Generalists into Specialists (NER & Classification)

# The limits of Prompt Engineering

- The Problem:
  - **Cost:** Sending long prompts to GPT-* is expensive at scale.
  - **Latency:** Large models are slow. A robot needs to react in milliseconds.
  - **Privacy:** You can't send sensitive data to a public API.
  - **Formatting:** Prompts sometimes fail to follow strict JSON schemas.
- **The Solution:** Fine-Tuning. Teaching a smaller, faster model to do *one thing* perfectly.

## Prompting

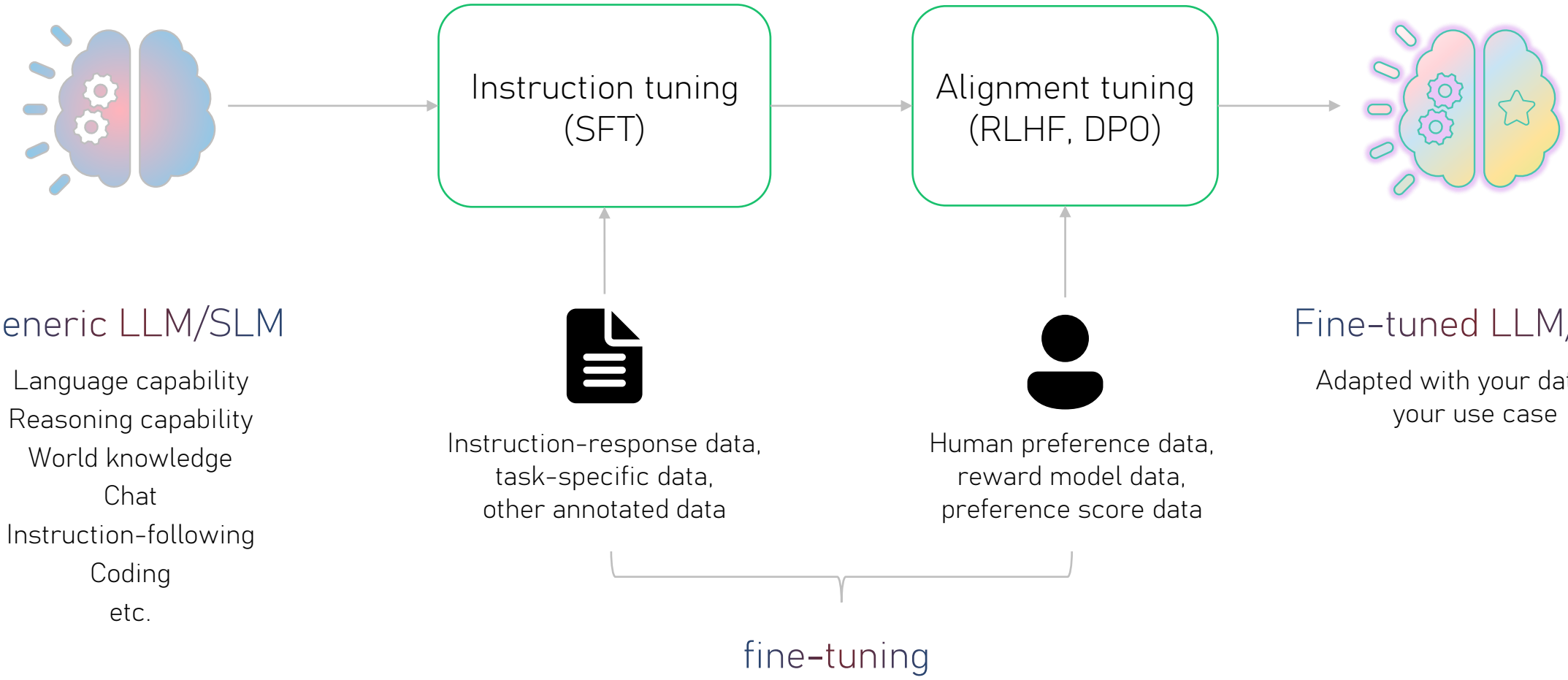
- Low barrier to entry
- Easy to make quick/small changes
- Enough for most jobs

## Fine-tuning

- Higher up-front investment
- Longer (more automated) iteration loops
- Specialized tool - efficiency & new capability

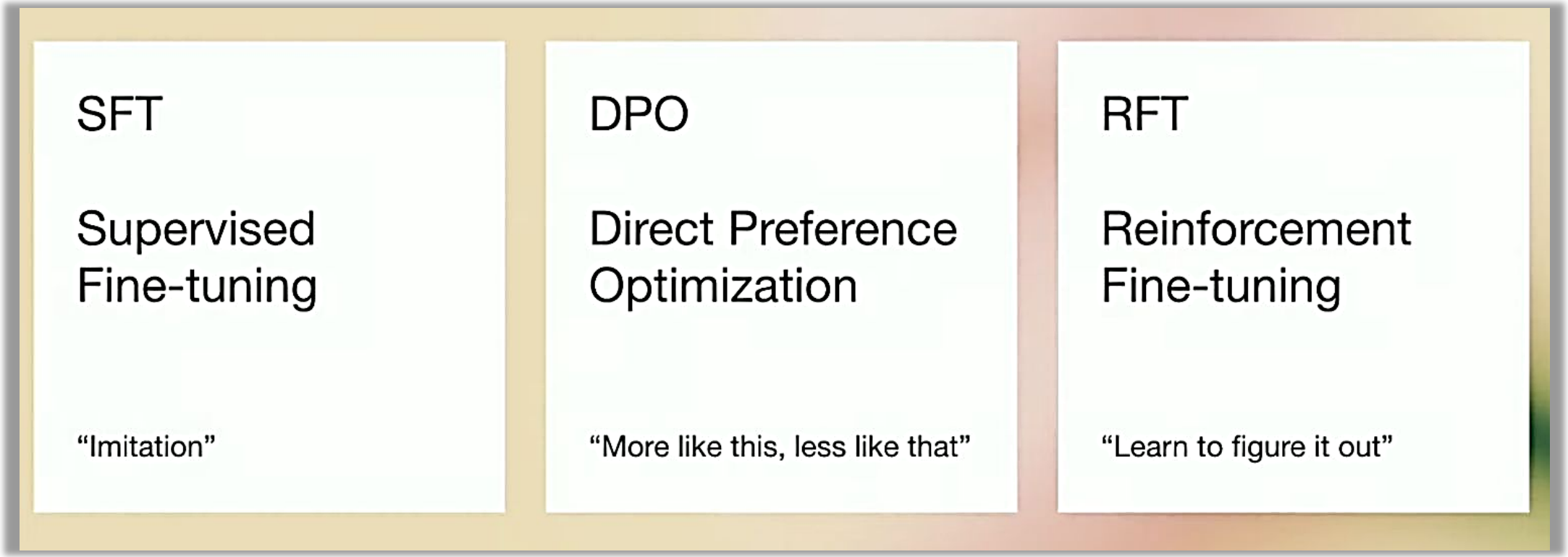
Image Credit: YT OpenAI: https://www.youtube.com/watch?v=JfaLQqfXqPA

# *Fine-tuning* a foundation model




BRK 150 deck: https://lnkd.in/ga7c2mj3 & Code: https://lnkd.in/g2-c-g_G

# SFT, DPO & RFT




YT OpenAI: https://www.youtube.com/watch?v=JfaLQqfXqPA
Reinforcement Fine-tuning: https://github.com/openai/openai-cookbook/tree/main/examples/fine-tuned_qa
Guide to DPO: https://github.com/openai/openai-cookbook/blob/main/examples/Fine_tuning_direct_preference_optimization_guide.ipynb

# Supervised Fine-tuning

Notes

- Best for “simple” tasks
- Model can regress on other tasks
- Data diversity is critical
- ~100 samples to start, 500+ is best

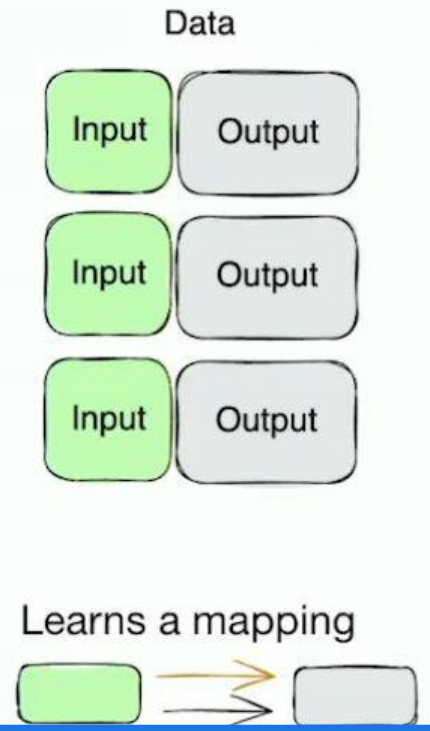



Image Credit: YT OpenAI: https://www.youtube.com/watch?v=JfaLQqfXqPA

# Direct Preference Optimization

Notes

- More forgiving of data diversity
- Made for explicit “which do you prefer” scenarios
- Best evals are “human evals” – same distribution as training
- Can be run after SFT

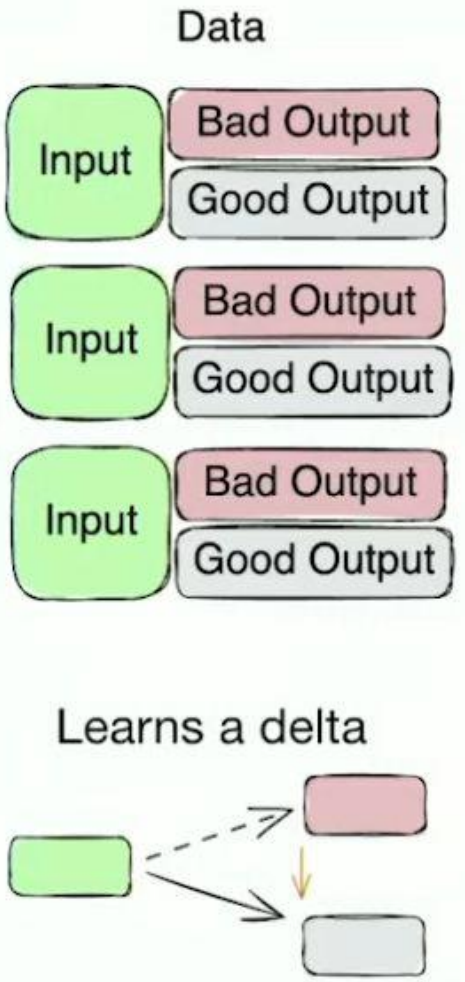



Image Credit: YT OpenAI: https://www.youtube.com/watch?v=JfaLQqfXqPA

# Reinforcement Fine-tuning

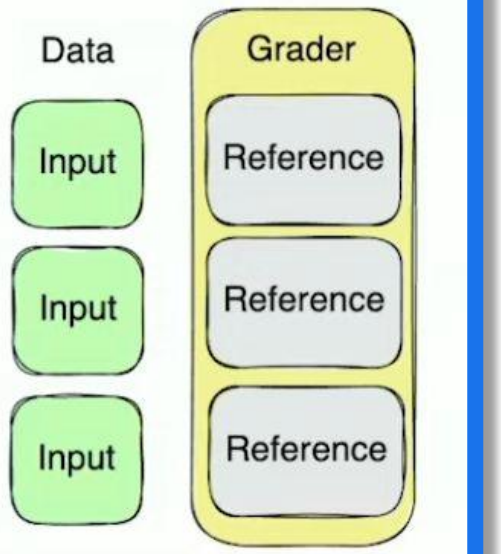
Data

- Unambiguous grading
- Very low noise, quality critical
- High signal
- 40 - 80, 150+ examples
- Prompt & context matter

Graders:

- String Check
- Text Similarity
- Python
- Score Model
- Label Model
- Multi

https://platform.openai.com/docs/api-reference/graders/string-check




Learns to reason

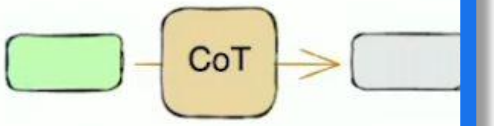
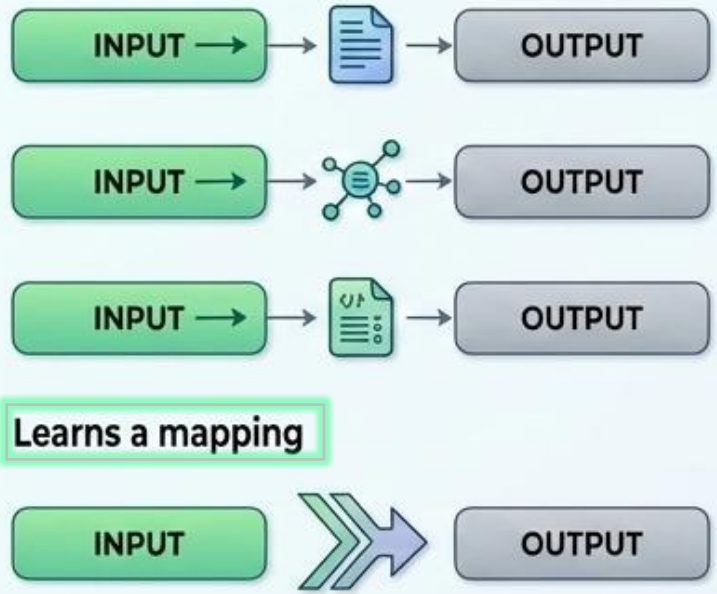



Image Credit: YT OpenAI: https://www.youtube.com/watch?v=JfaLQqfXqPA

# ADVANCED LLM FINE-TUNING TECHNIQUES

## SUPERVISED FINE-TUNING (SFT)

### Data Structure

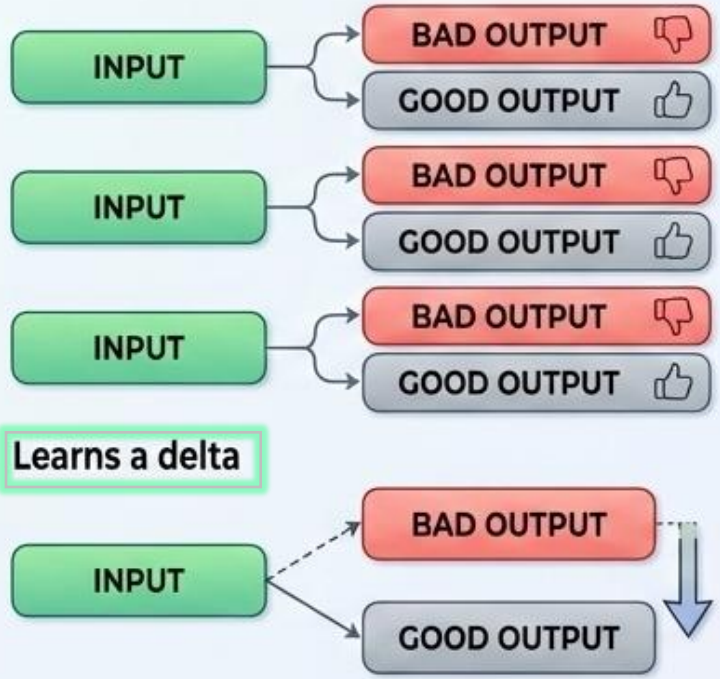



### CONSTRAINED, DESIRED OUTPUTS

- **CLASSIFICATION** (e.g., Sentiment Analysis)
- **FORMATTING** (e.g., JSON, Markdown)
- **STRUCTURED EXTRACTION** (e.g., Entities, Relations)

## DIRECT PREFERENCE OPTIMIZATION (DPO)

### Data Structure

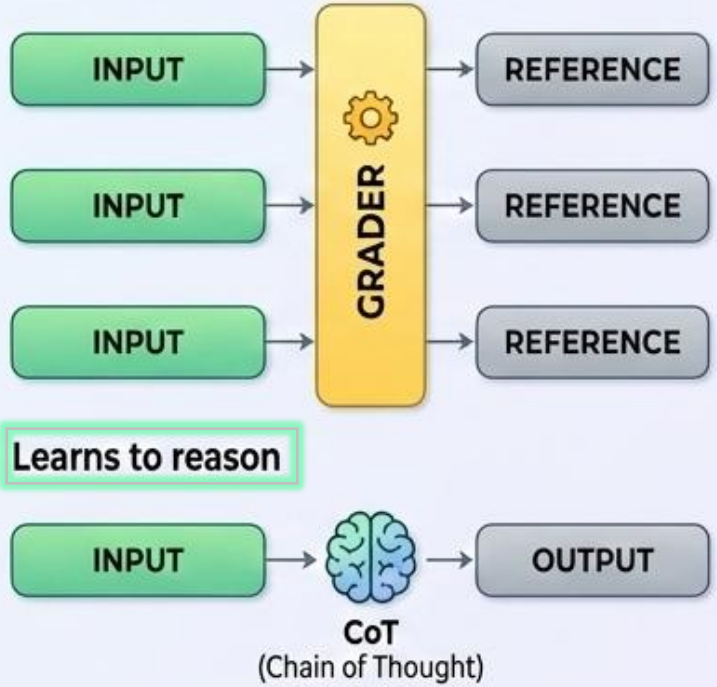



### CONTRASTIVE IMPROVEMENT

- **TONE MATCHING** (e.g., Polite, Professional)
- **INTERNALIZING A/B TESTS** (e.g., User Preferences)

## REINFORCEMENT FINE-TUNING (RFT)

### Data Structure




### GRADABLE, HARD PROBLEMS

- **MEDICAL, LEGAL, CODING** (e.g., Complex Reasoning)
- **UNAMBIGUOUS SOLUTIONS** (e.g., Math, Logic Puzzles)
- **LLM AS A JUDGE MODEL** (e.g., Evaluating Quality)

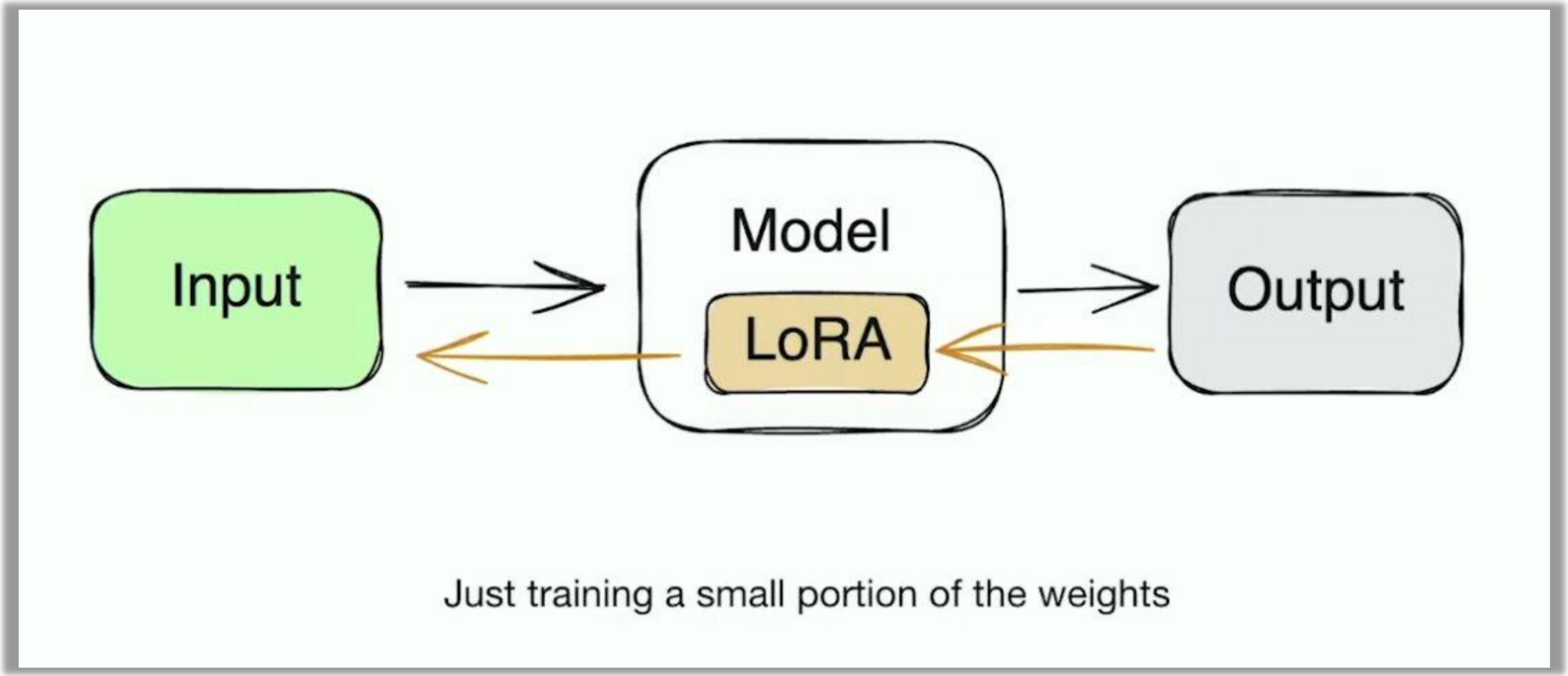
Image Credit: YT OpenAI: https://www.youtube.com/watch?v=JfaLQqfXqPA recreated with Nano Banana Pro

## Comparison of SFT, DPO, and RFT

| Feature | SFT (Supervised Fine-Tuning) | DPO (Direct Preference Optimization) | RFT (Reinforcement Fine-Tuning) |
|---|---|---|---|
| **Core Concept** | Trains a model on specific input-output pairs to **imitate** the desired output. | Trains a model by showing it pairs of **'preferred'** and **'non-preferred'** outputs for a given input. | Trains a model using reinforcement learning to generate outputs that maximize a **score from a 'grader'**. |
| **Learning Mechanism** | Imitation | Preference Optimization | Reward Maximization |
| **Required Data** | A dataset of high-quality, specific input-output pairs (prompt and ideal completion). | A dataset where each input has a 'preferred' (chosen) output and a 'non-preferred' (rejected) output. | A dataset of inputs and a programmatic 'grader' (e.g., Python code, LLM-as-judge) that scores the model's generated outputs. |
| **Best Suited For** | Constrained tasks with a single correct answer (e.g., classification, formatting, structured data extraction), and model distillation. | Subjective tasks where 'better' is easier to define than 'perfect' (e.g., matching tone/style, improving helpfulness). Ideal for data from A/B tests. | Complex reasoning tasks that require a chain of thought or logical steps to arrive at a correct answer. Can achieve state-of-the-art results. |
| **Key Advantages** | • Simplicity.<br>• Effective for constrained tasks.<br>• Learns a direct mapping from input to output. | • More forgiving on data diversity.<br>• Excellent for matching tone and style.<br>• Leverages human preference (A/B) data well. | • Can achieve state-of-the-art performance.<br>• Data efficient with high-signal examples.<br>• Learns the reasoning process, not just the answer. |
| **Key Disadvantages** | • Data quality and diversity are critical.<br>• Can overfit to the training data.<br>• May "regress" or perform worse on tasks outside the training set. | • Can be harder to evaluate objectively.<br>• Output is not guaranteed to be exact, just "more preferred".<br>• Still requires good prompting. | • Highly sensitive to noisy or inconsistent data.<br>• Requires a complex, unambiguous, and well-defined grader.<br>• Can overfit to the grader's logic. |

# A look ahead on PEFT (Parameter-Efficient Fine-Tuning) & LORA (Low-Rank Adaptation)

- *Bridge to future modules.*
- **Question:** What if the model is too big to fine-tune on my laptop (like Llama 3)?
- **Preview:** We don't tune all parameters. We use **PEFT (Parameter-Efficient Fine-Tuning)** to tune only 0.1% of the weights. (We will discuss this in advanced Generative AI modules).



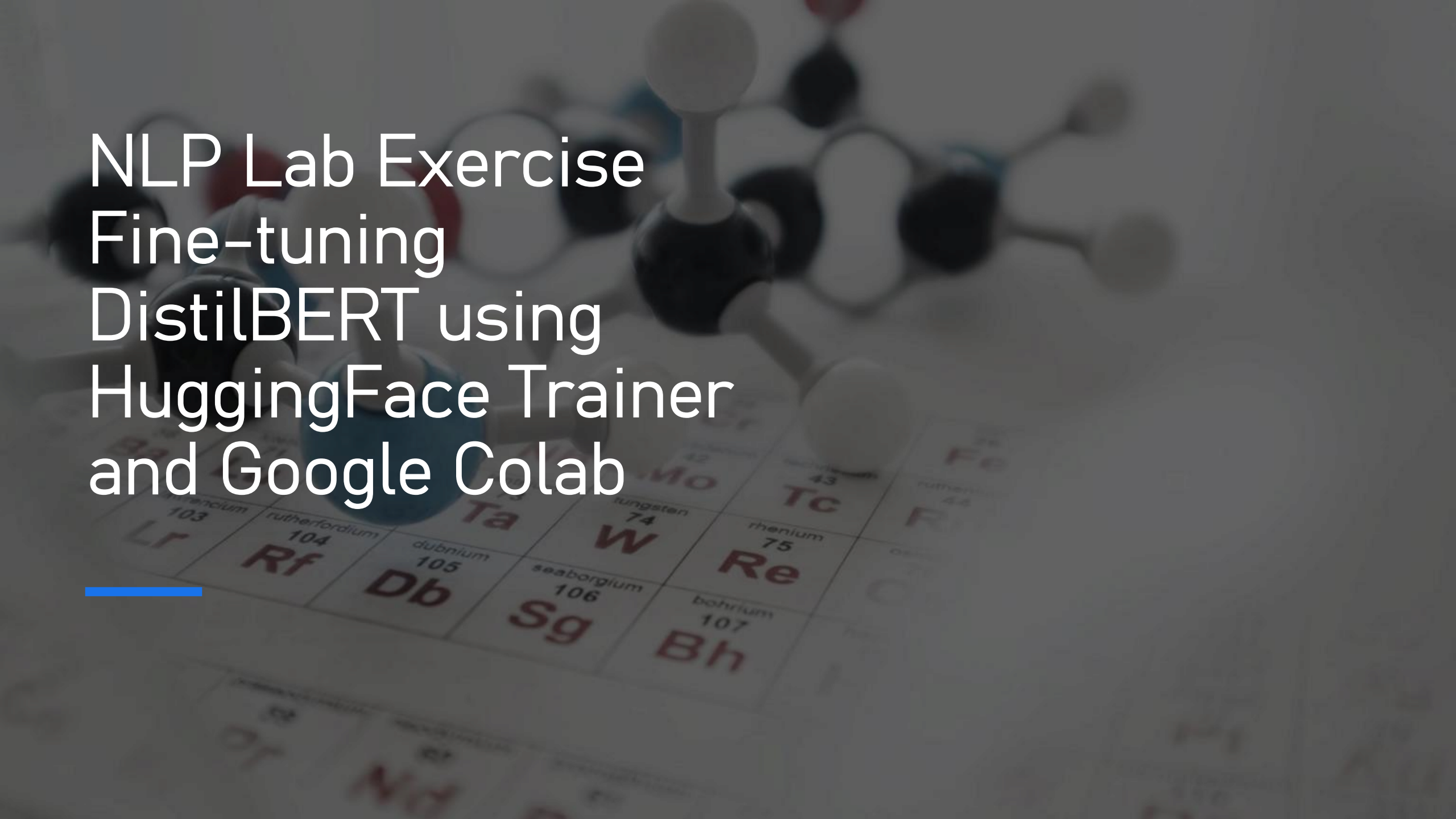

Microsoft arxiv paper on LoRA: https://arxiv.org/pdf/2106.09685

Image Credit: YT OpenAI: https://www.youtube.com/watch?v=JfaLQqfXqPA

# Distillation vs Fine-tuning

- **Knowledge Distillation**:
  A Student Model learns from a teacher model by mimicking its outputs (often soft logits or intermediate representation).
  **Goal**: Compress knowledge from a larger model into smaller one while preserving performance.
- **Typical Use case**: Model Size Reduction

- **Fine-Tuning (RFT/DPO/SFT)**:
  These Adjust the weights of a model using labeled data or preference signals.
  **Goal**: Align the model with desired behaviors or domain-specific tasks. They do not involve transferring knowledge from a teacher to student model.
- **Typical Use case**: Refer earlier slides.




# NLP Lab Exercise Fine-tuning DistilBERT using HuggingFace Trainer and Google Colab

# Fine-Tuning a Model in Google Colab